

```

import logging
import json
import os
from datetime import datetime
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup, ReplyKey
from telegram.ext import Application, CommandHandler, MessageHandler, CallbackQue
from telegram.constants import ParseMode

logging.basicConfig(format="%(asctime)s - %(levelname)s - %(message)s", level=log
logger = logging.getLogger(__name__)

BOT_TOKEN = os.environ.get("BOT_TOKEN", "8772939585:AAG4izepyO-4H1Kzk_npbagFhO80A
ADMIN_CHAT_ID = int(os.environ.get("ADMIN_CHAT_ID", "474705465"))
KASPI_QR_URL = "https://pay.kaspi.kz/pay/rrpncedw"
DB_FILE = "applications.json"

PLANS = {
    "single": {
        "name": "Razovaya poyezdka",
        "emoji": "🚗",
        "price": "3 000 T",
        "price_int": 3000,
        "desc": "do 8 km",
        "features": ["Detskoe kreslo", "Professional voditel", "Podacha 7-10 min"
    },
    "priority": {
        "name": "Mama Priority",
        "emoji": "★",
        "price": "45 000 T",
        "price_int": 45000,
        "desc": "primerno 16 poyezdok v mesyac",
        "features": ["Vse iz Razovoy", "Prioritetnaya podacha", "Skidka 15%", "Be
    },
    "family": {
        "name": "Semeynyy",
        "emoji": "👨‍👩‍👧",
        "price": "60 000 T",
        "price_int": 60000,
        "desc": "do 24 poyezdok v mesyac",
        "features": ["Vse iz Mama Priority", "2+ detskih kresla", "Miniven po zap
    }
}

ASK_NAME, ASK_PHONE, ASK_PLAN, CONFIRM_ORDER, PAYMENT_CONFIRM = range(5)

```

```

def load_db():
    if os.path.exists(DB_FILE):
        with open(DB_FILE, "r", encoding="utf-8") as f:
            return json.load(f)
    return {"applications": [], "counter": 0}

def save_db(data):
    with open(DB_FILE, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def add_application(app):
    db = load_db()
    db["counter"] += 1
    app["id"] = db["counter"]
    app["created_at"] = datetime.now().strftime("%d.%m.%Y %H:%M")
    app["status"] = "new"
    db["applications"].append(app)
    save_db(db)
    return app["id"]

def get_applications(status=None):
    db = load_db()
    apps = db["applications"]
    if status:
        apps = [a for a in apps if a.get("status") == status]
    return apps

def update_application_status(app_id, status):
    db = load_db()
    for app in db["applications"]:
        if app["id"] == app_id:
            app["status"] = status
            break
    save_db(db)

def delete_application(app_id):
    db = load_db()
    before = len(db["applications"])
    db["applications"] = [a for a in db["applications"] if a["id"] != app_id]
    save_db(db)
    return len(db["applications"]) < before

def is_admin(user_id):
    return user_id == ADMIN_CHAT_ID

def main_keyboard():

```

```

return ReplyKeyboardMarkup([
    [KeyboardButton("Ostavit zayavku")],
    [KeyboardButton("Tarify"), KeyboardButton("O servise")]
], resize_keyboard=True)

def admin_keyboard():
    return ReplyKeyboardMarkup([
        [KeyboardButton("Vse zayavki"), KeyboardButton("Novye zayavki")],
        [KeyboardButton("Oplachennye"), KeyboardButton("Statistika")]
    ], resize_keyboard=True)

def plan_keyboard():
    buttons = []
    for key, plan in PLANS.items():
        buttons.append([InlineKeyboardButton(
            f"{plan['emoji']} {plan['name']} - {plan['price']}", callback_data=f"
        )])
    return InlineKeyboardMarkup(buttons)

def phone_keyboard():
    return ReplyKeyboardMarkup([
        [KeyboardButton("Otpavit nomer telefona", request_contact=True)],
        [KeyboardButton("Otmena")]
    ], resize_keyboard=True, one_time_keyboard=True)

def payment_keyboard():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("Oplatit cherez Kaspi QR", url=KASPI_QR_URL)],
        [InlineKeyboardButton("Ya oplatil(a)", callback_data="paid")],
        [InlineKeyboardButton("Otmenit zayavku", callback_data="cancel_order")]
    ])

def app_actions_keyboard(app_id, status):
    buttons = []
    if status != "paid":
        buttons.append([InlineKeyboardButton("Otmetit oplachennym", callback_data
        buttons.append([InlineKeyboardButton("Otmenit", callback_data=f"cancel_app_{a
        buttons.append([InlineKeyboardButton("Udalit zayavku", callback_data=f"delete
    return InlineKeyboardMarkup(buttons)

def format_app(app):
    plan = PLANS.get(app.get("plan", ""), {})
    status_map = {"new": "Novaya", "payment_sent": "Zhdet podtverzhdeniya", "paid
    return (
        f"Zayavka #{app['id']}\n"
        f"Imya: {app['name']}\n"
        f"Telefon: {app['phone']}\n"

```

```

        f"Tarif: {plan.get('emoji','')} {plan.get('name', app.get('plan',''))}\n"
        f"Summa: {plan.get('price', '')}\n"
        f>Data: {app['created_at']}\n"
        f>Status: {status_map.get(app.get('status','new'), app.get('status',''))}
    )

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user = update.effective_user
    if is_admin(user.id):
        await update.message.reply_text(
            "Dobro pozhalovat, Administrator!\nPanel upravleniya zayavkami MamaGo
            reply_markup=admin_keyboard()
        )
        return ConversationHandler.END
    await update.message.reply_text(
        f"Privet, {user.first_name}!\n\n"
        "Dobro pozhalovat v MamaGo – bezopasnoye taxi dlya mam s detmi.\n\n"
        "Nazhmite 'Ostavit zayavku' chtoby zapisatsya.",
        reply_markup=main_keyboard()
    )
    return ConversationHandler.END

async def show_about(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "O servise MamaGo:\n\n"
        "Bezopasnoye taxi dlya mam s detmi.\n"
        "Detskie kresla v kazhdoy mashine.\n"
        "Proverennye voditeli.\n"
        "Podacha 7-10 minut.\n\n"
        "Vazhno: my vozim detey TOLKO vmeste s roditelyami ili soprovozhdayushchi
        reply_markup=main_keyboard()
    )

async def show_plans(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = "Nashi tarify:\n\n"
    for key, plan in PLANS.items():
        text += f"{plan['emoji']} {plan['name']} – {plan['price']}\n"
        text += f"{plan['desc']}\n"
        for f in plan['features']:
            text += f"  + {f}\n"
        text += "\n"
    await update.message.reply_text(text, reply_markup=main_keyboard())

async def start_application(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data.clear()
    await update.message.reply_text(
        "Oformlenie zayavki\n\nShag 1 iz 3\n\nKak vas zovut? Vvedite imya i famil

```

```

        reply_markup=ReplyKeyboardMarkup([["Otmena"]], resize_keyboard=True)
    )
    return ASK_NAME

async def got_name(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text.strip()
    if text == "Otmena":
        await update.message.reply_text("Zayavka otmenena.", reply_markup=main_ke
        return ConversationHandler.END
    if len(text) < 2:
        await update.message.reply_text("Pozhaluysta, vvedite vashe imya (minimum
        return ASK_NAME
    context.user_data["name"] = text
    await update.message.reply_text(
        f"Otlichno, {text}!\n\nShag 2 iz 3\n\nUkazhite vash nomer telefona:",
        reply_markup=phone_keyboard()
    )
    return ASK_PHONE

async def got_phone_contact(update: Update, context: ContextTypes.DEFAULT_TYPE):
    phone = update.message.contact.phone_number
    if not phone.startswith("+"):
        phone = "+" + phone
    context.user_data["phone"] = phone
    await update.message.reply_text("Shag 3 iz 3\n\nVyberite tarif:", reply_marku
    return ASK_PLAN

async def got_phone_text(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text.strip()
    if text == "Otmena":
        await update.message.reply_text("Zayavka otmenena.", reply_markup=main_ke
        return ConversationHandler.END
    phone = text.replace(" ", "").replace("-", "")
    if len(phone) < 10:
        await update.message.reply_text("Vvedite korektnyy nomer telefona (naprim
        return ASK_PHONE
    context.user_data["phone"] = phone
    await update.message.reply_text("Shag 3 iz 3\n\nVyberite tarif:", reply_marku
    return ASK_PLAN

async def got_plan(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    plan_key = query.data.replace("plan_", "")
    plan = PLANS.get(plan_key)
    if not plan:
        return ASK_PLAN

```

```

context.user_data["plan"] = plan_key
name = context.user_data["name"]
phone = context.user_data["phone"]
text = (
    f"Proverite zayavku:\n\n"
    f"Imya: {name}\n"
    f"Telefon: {phone}\n"
    f"Tarif: {plan['emoji']} {plan['name']}\n"
    f"Summa: {plan['price']}\n\n"
    f"Vse verno?"
)
keyboard = InlineKeyboardMarkup([
    [InlineKeyboardButton("Podtverdit", callback_data="confirm_yes")],
    [InlineKeyboardButton("Izmenit", callback_data="confirm_no")]
])
await query.edit_message_text(text, reply_markup=keyboard)
return CONFIRM_ORDER

async def confirm_order(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    if query.data == "confirm_no":
        await query.edit_message_text("Vvedite vashe imya:")
        await query.message.reply_text("Vvedite vashe imya:", reply_markup=ReplyK
        return ASK_NAME
    app = {
        "user_id": update.effective_user.id,
        "name": context.user_data["name"],
        "phone": context.user_data["phone"],
        "plan": context.user_data["plan"],
    }
    app_id = add_application(app)
    plan = PLANS[context.user_data["plan"]]
    admin_text = (
        f"NOVAYA ZAYAVKA!\n\n"
        f"Zayavka #{app_id}\n"
        f"Imya: {app['name']}\n"
        f"Telefon: {app['phone']}\n"
        f"Tarif: {plan['emoji']} {plan['name']}\n"
        f"Summa: {plan['price']}"
    )
    try:
        await context.bot.send_message(chat_id=ADMIN_CHAT_ID, text=admin_text, re
    except Exception as e:
        logger.error(f"Admin notification error: {e}")
    payment_text = (
        f"Zayavka #{app_id} prinyata!\n\n"

```

```

        f"Tarif: {plan['emoji']} {plan['name']}\n"
        f"K oplate: {plan['price']}\n\n"
        f"Oplata cherez Kaspi QR:\n"
        f"1. Nazhmite 'Oplatit cherez Kaspi QR'\n"
        f"2. Otskaniruyte QR-kod v prilozhenii Kaspi\n"
        f"3. Vvedite summu: {plan['price']}\n"
        f"4. Vernitesya syuda i nazhmite 'Ya oplatil(a)'"
    )
    await query.edit_message_text(payment_text, reply_markup=payment_keyboard())
    return PAYMENT_CONFIRM

async def payment_done(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    if query.data == "cancel_order":
        await query.edit_message_text("Zayavka otmenena.")
        await query.message.reply_text("Glavnoe menu:", reply_markup=main_keyboard())
        return ConversationHandler.END
    db = load_db()
    for app in reversed(db["applications"]):
        if app.get("user_id") == update.effective_user.id and app.get("status") != "payment_sent":
            app["status"] = "payment_sent"
            break
    save_db(db)
    await query.edit_message_text("Spasibo! Oplata na proverke.\n\nNash menedzher")
    await query.message.reply_text("Glavnoe menu:", reply_markup=main_keyboard())
    try:
        await context.bot.send_message(chat_id=ADMIN_CHAT_ID, text=f"Polzovatel {update.effective_user.username} zaplatil")
    except Exception as e:
        logger.error(e)
    return ConversationHandler.END

async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Otmeneno.", reply_markup=main_keyboard())
    return ConversationHandler.END

async def admin_all_apps(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not is_admin(update.effective_user.id):
        return
    apps = get_applications()
    if not apps:
        await update.message.reply_text("Zayavok poka net.", reply_markup=admin_keyboard())
        return
    await update.message.reply_text(f"Vse zayavki ({len(apps)}):", reply_markup=admin_keyboard())
    for app in apps[-20:]:
        await update.message.reply_text(format_app(app), reply_markup=app_actions[app['status']])

```

```

async def admin_new_apps(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not is_admin(update.effective_user.id):
        return
    apps = get_applications(status="new")
    if not apps:
        await update.message.reply_text("Novykh zayavok net.", reply_markup=admin
        return
    await update.message.reply_text(f"Novye zayavki ({len(apps)}):", reply_markup
    for app in apps:
        await update.message.reply_text(format_app(app), reply_markup=app_actions

async def admin_paid_apps(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not is_admin(update.effective_user.id):
        return
    apps = get_applications(status="paid")
    if not apps:
        await update.message.reply_text("Oplachennykh zayavok net.", reply_markup
        return
    await update.message.reply_text(f"Oplachennye zayavki ({len(apps)}):", reply_
    for app in apps:
        await update.message.reply_text(format_app(app), reply_markup=app_actions

async def admin_stats(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not is_admin(update.effective_user.id):
        return
    db = load_db()
    apps = db["applications"]
    total = len(apps)
    new_count = sum(1 for a in apps if a.get("status") == "new")
    pending = sum(1 for a in apps if a.get("status") == "payment_sent")
    paid = sum(1 for a in apps if a.get("status") == "paid")
    cancelled = sum(1 for a in apps if a.get("status") == "cancelled")
    revenue = sum(PLANS.get(a.get("plan", ""), {}).get("price_int", 0) for a in ap
    text = (
        f"Statistika MamaGo\n\n"
        f"Vsego zayavok: {total}\n"
        f"Novye: {new_count}\n"
        f"Zhдут podtverzheniya: {pending}\n"
        f"Oplacheny: {paid}\n"
        f"Otmeneny: {cancelled}\n\n"
        f"Vyruchka: {revenue:,} T".replace(",", " ")
    )
    await update.message.reply_text(text, reply_markup=admin_keyboard())

async def admin_callback(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    if not is_admin(query.from_user.id):

```



```

        await query.answer("Net dostupa")
        return
    await query.answer()
    data = query.data
    if data.startswith("mark_paid_"):
        app_id = int(data.replace("mark_paid_", ""))
        update_application_status(app_id, "paid")
        await query.edit_message_text(query.message.text + "\n\nOtmetyeno kak opla
    elif data.startswith("cancel_app_"):
        app_id = int(data.replace("cancel_app_", ""))
        update_application_status(app_id, "cancelled")
        await query.edit_message_text(query.message.text + "\n\nZayavka otmenena"
    elif data.startswith("delete_app_"):
        app_id = int(data.replace("delete_app_", ""))
        deleted = delete_application(app_id)
        if deleted:
            await query.edit_message_text(f"Zayavka #{app_id} udalena.")
        else:
            await query.answer("Zayavka ne naydena")

def main():
    application = Application.builder().token(BOT_TOKEN).build()
    conv = ConversationHandler(
        entry_points=[MessageHandler(filters.Regex("^Ostavit zayavku$"), start_ap
        states={
            ASK_NAME: [MessageHandler(filters.TEXT & ~filters.COMMAND, got_name)]
            ASK_PHONE: [
                MessageHandler(filters.CONTACT, got_phone_contact),
                MessageHandler(filters.TEXT & ~filters.COMMAND, got_phone_text)
            ],
            ASK_PLAN: [CallbackQueryHandler(got_plan, pattern="^plan_")],
            CONFIRM_ORDER: [CallbackQueryHandler(confirm_order, pattern="^confirm
            PAYMENT_CONFIRM: [CallbackQueryHandler(payment_done, pattern="^(paid|
        ],
        fallbacks=[CommandHandler("cancel", cancel)],
        allow_reentry=True
    )
    application.add_handler(conv)
    application.add_handler(CommandHandler("start", start))
    application.add_handler(MessageHandler(filters.Regex("^O servise$"), show_abo
    application.add_handler(MessageHandler(filters.Regex("^Tarify$"), show_plans)
    application.add_handler(MessageHandler(filters.Regex("^Vse zayavki$"), admin_
    application.add_handler(MessageHandler(filters.Regex("^Novye zayavki$"), admi
    application.add_handler(MessageHandler(filters.Regex("^Oplachennye$"), admin_
    application.add_handler(MessageHandler(filters.Regex("^Statistika$"), admin_s
    application.add_handler(CallbackQueryHandler(admin_callback, pattern="^(mark_
    logger.info("MamaGo bot started!")

```

```
application.run_polling(drop_pending_updates=True)
```

```
if __name__ == "__main__":  
    main()
```